

基于 Socket 的 Web 代理服务器设计与实现

计算机保密课程期末实验报告

学生姓名：_____

学 号：_____

专业班级：_____

指导教师：_____

2026 年 6 月

摘要

本实验围绕 Web 代理服务器的设计与实现展开，使用 Python 的 Socket、线程和标准库 HTTP 服务组件构建正向代理。系统能够解析客户端代理请求，将普通 HTTP 请求转发给目标服务器，并把服务器响应返回给客户端；能够按照域名、URL 关键词、请求方法、客户端 IP 地址或 CIDR 网段执行访问控制；能够检查明文 HTTP 文本响应中的指定关键词，并在命中后生成带有拦截原因的提示页面。

在基础要求之上，项目增加了 HTTPS CONNECT 隧道、HTTP GET 缓存、代理认证、访问频率限制、黑白名单模式、动态规则管理、运行统计、日志审计和管理前端。规则可以通过网页、JSON API 或命令行工具进行增删改查，修改后立即作用于后续请求，并持久化写回 JSON 配置文件。日志模块记录访问、拦截、错误和规则变更信息，支持按事件与关键词查询，并可导出 CSV。项目使用分批自动测试和逐模块手工验收相结合的方式进行验证，完整回归覆盖 15 项 Web/代理功能测试和 7 项规则与运行模式测试。

关键词：Web 代理；Socket；访问控制；关键词过滤；日志审计；动态规则

目录

1	实验背景与任务分析	5
1.1	实验题目	5
1.2	功能需求	5
1.3	运行环境与技术选型	6
2	系统总体设计	6
2.1	总体架构	6
2.2	源码结构	6
2.3	请求处理流程	7
2.4	规则与配置模型	7
3	核心功能设计与实现	8
3.1	HTTP 请求解析与代理转发	8
3.2	域名、URL 与请求方法访问控制	9
3.3	网页正文关键词过滤	9
3.4	HTTPS CONNECT 隧道与可见性边界	9
3.5	客户端 IP 与 CIDR 访问控制	10
3.6	代理认证与访问频率限制	10
3.7	HTTP GET 缓存	10
3.8	动态规则管理与持久化	10
3.9	日志审计与运行统计	11
3.10	管理前端	11
4	测试方案与验收结果	12
4.1	测试方法	12
4.2	自动测试结果	12
4.3	客户端 IP 黑白名单实测	12
4.4	核心功能手工验收	13
5	安全性分析与项目边界	13

5.1 安全控制效果	13
5.2 现有边界	14
 6 总结与展望	 14
 A 常用运行与验收命令	 14
 B 主要管理 API	 15

1 实验背景与任务分析

1.1 实验题目

本实验题目为“设计和实现一个 Web 代理服务器”，具体要求包括：能够转发客户端的 Web 访问请求给服务器并把返回的网页返回给客户端；能够拦截指定域名的访问请求并过滤指定关键词的网页；开发语言不限。

Web 代理位于客户端与目标服务器之间。客户端将访问请求发送给代理，代理根据请求中的目标地址建立上游连接，再把目标服务器的响应交还给客户端。由于代理参与请求和响应的传递过程，它能够在转发前执行访问控制，也能够在收到明文响应后执行内容检查。该结构适合用于理解网络请求解析、访问策略、内容过滤和安全审计的基本原理。

1.2 功能需求

根据题目要求和课程展示需要，项目功能分为基础功能、管理功能和扩展功能三类。基础功能直接对应题目要求，包括 HTTP 代理转发、域名拦截和网页正文关键词过滤。管理功能用于保证规则能够被清楚地配置、查询和验证，包括规则增删改查、运行统计、日志查询和前端展示。扩展功能用于完善代理服务器的实际使用能力，包括 HTTPS CONNECT、客户端 IP 访问控制、代理认证、访问频率限制、HTTP GET 缓存和黑白名单模式。

功能	需求说明
HTTP 代理转发	接收客户端代理请求，连接目标 Web 服务器，将请求和响应完整转发。
域名访问控制	按完整域名、父域名后缀或显式通配符拦截目标域名。
正文关键词过滤	对明文 HTTP 文本响应进行关键词检查，命中后阻止原文返回。
URL 与方法过滤	按请求路径、查询参数和 HTTP 方法执行拒绝策略。
客户端访问控制	支持单个 IPv4、IPv6 地址和 CIDR 网段的黑名单与白名单。
动态规则管理	规则支持新增、删除、修改、查询和整组替换，运行期间立即生效。

功能	需求说明
日志与统计	记录访问、拦截、错误和规则变更，提供统计、查询和 CSV 导出。
管理前端	在浏览器中显示规则、运行状态、日志、统计和变更记录。

1.3 运行环境与技术选型

实验运行环境为 Windows、Windows PowerShell、Visual Studio Code 和 Python 3。项目主要使用 Python 标准库完成，不依赖额外 Web 框架。底层代理服务使用 `socket` 建立 TCP 监听和上游连接，使用 `threading` 为客户端连接提供并发处理，使用 `select` 完成 HTTPS CONNECT 隧道的双向数据转发。管理服务使用 `http.server.ThreadingHTTPServer` 提供静态前端页面和 JSON API。

配置与规则使用 JSON 文件保存，规则变更使用 JSONL 逐行记录，运行日志使用结构化文本格式写入。IP 地址和 CIDR 网段由 `ipaddress` 模块负责解析和标准化，域名显式通配符由 `fnmatch` 完成匹配。管理前端使用原生 HTML、CSS 和 JavaScript，能够直接通过浏览器调用管理 API。

2 系统总体设计

2.1 总体架构

系统由代理服务、管理后端、管理前端、配置与日志文件、测试工具五部分组成。代理服务监听 127.0.0.1:8080，负责处理客户端流量；管理后端监听 127.0.0.1:8088，负责提供前端页面和管理接口。两个端口共享同一个运行状态对象，规则修改后可以直接影响代理服务的新请求。

2.2 源码结构

后端入口文件为 `src/proxy.py`，其中包含运行状态、HTTP 请求解析、访问策略链、普通 HTTP 转发、HTTPS CONNECT、管理 API 和服务器启动逻辑。配置与规则基础能力拆分到 `src/webproxy/config_rules.py`，日志查询与 CSV 导出拆分到 `src/webproxy/audit.py`，自动验收证据读取与统计重建拆分到 `src/webproxy/evidence.py`。管理前端由 `frontend/index.html`、`frontend/logs.html` 和 `frontend/changes.html` 组成。

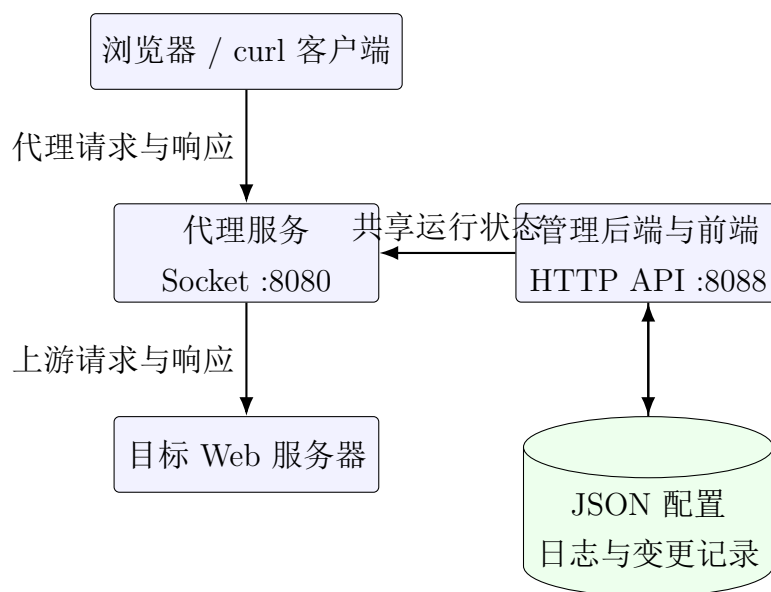


图 1: 系统总体架构

2.3 请求处理流程

代理接收到客户端连接后，首先读取请求头并解析请求方法、目标主机、目标端口、路径和请求头。随后按固定顺序执行客户端访问控制、代理认证、访问频率限制、方法规则、域名规则和 URL 规则。被策略拒绝的请求直接返回 403、407 或 429 响应，并写入日志。

通过前置策略检查后，GET 请求会查询缓存。缓存命中时直接返回已保存的响应；缓存未命中时继续连接上游服务器。普通 HTTP 响应在返回客户端之前执行正文关键词检查，HTTPS CONNECT 请求则建立双向 TCP 隧道。处理结束后，系统根据结果更新统计、排行与日志。

2.4 规则与配置模型

项目使用 JSON 保存规则和运行参数。列表型规则包括域名黑名单、域名白名单、客户端 IP 黑名单、客户端 IP 白名单、URL 关键词、正文关键词和禁止方法。运行设置包括黑白名单模式、缓存开关与容量、认证开关与用户表、限流开关与时间窗口等。

运行时由 `RuntimeState` 保存当前配置、统计、缓存、限流桶、访问排行和规则变更历史。共享状态的读写使用线程锁保护。规则修改成功后，内存数据与 JSON 配置同步更新，后续请求读取更新后的配置。

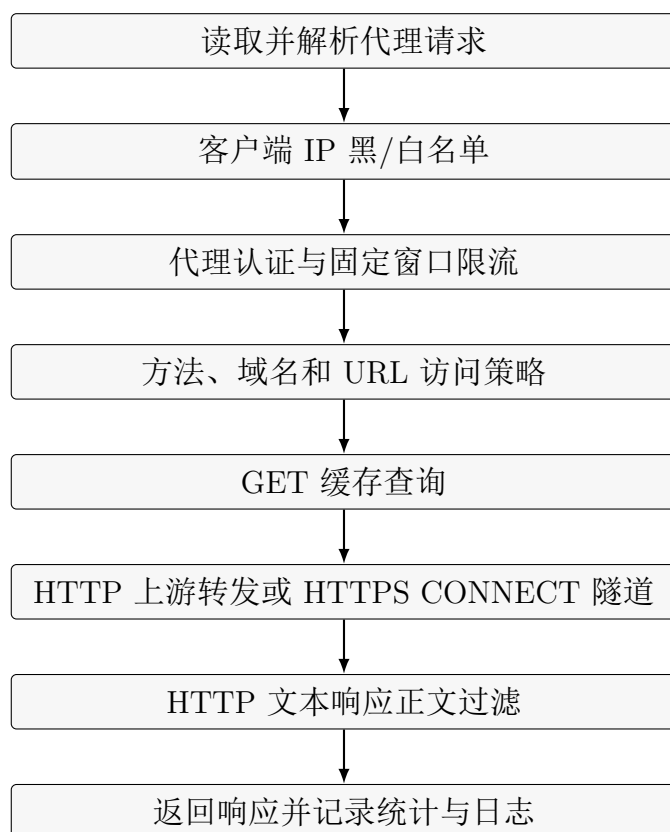


图 2: 代理请求处理流程

3 核心功能设计与实现

3.1 HTTP 请求解析与代理转发

浏览器向 HTTP 代理发送请求时，请求行通常携带完整 URL，例如：

```
GET http://127.0.0.1:9000/index.html HTTP/1.1
Host: 127.0.0.1:9000
```

`parse_http_request()` 根据请求方法和目标格式提取主机、端口与路径。普通 HTTP 请求默认使用 80 端口；CONNECT 请求默认使用 443 端口。解析失败或缺少目标主机时，代理返回 400 Bad Request。

目标服务器接收的请求行只需要路径，因此 `build_upstream_request()` 将代理格式请求改写为普通服务器格式，并重新生成 Host 头。代理会移除连接级请求头和代理认证头，设置 `Connection: close`，同时设置 `Accept-Encoding: identity`，便于读取完整响应和检查明文正文。

`forward_http()` 使用 `socket.create_connection()` 连接目标服务器，发送改写后的请求，并循环读取响应字节。读取结束后，响应经过正文过滤，再发送给客户端。

上游连接超时返回 504 Gateway Timeout，连接失败返回 502 Bad Gateway。

3.2 域名、URL 与请求方法访问控制

域名规则由 `domain_matches()` 执行。规则支持完整域名、父域名后缀和显式通配符。规则 `baidu.com` 可以匹配该域名及其子域名；规则 `*.baidu.com` 使用通配符匹配子域名；规则 `*baidu*` 执行范围更宽的包含匹配。普通短词不会自动扩展为任意包含匹配，从而减少误拦截。

`check_access_policy()` 按请求方法、白名单模式、域名黑名单和 URL 关键词的顺序检查目标请求。URL 搜索内容由目标主机、路径和原始请求目标组成，统一转换为小写后执行关键词包含判断。命中规则时函数返回拒绝结果和明确原因，例如 `domain_blacklist`、`method_blacklist` 或 `url_keyword:game`。

3.3 网页正文关键词过滤

正文过滤由 `filter_response_content()` 完成。函数先分离响应头和响应体，再检查 Content-Type 与 Content-Encoding。只有 HTML、纯文本、CSS、JavaScript 和 JSON 等文本类型进入关键词检查；压缩响应和二进制响应直接放行。

检查时将 UTF-8 响应正文转换为小写，并依次匹配 `blocked_content_keywords` 规则。命中关键词后，系统停止返回原始网页内容，调用 `build_policy_block_response()` 生成 403 Forbidden 页面。页面显示拦截原因、命中规则、请求方法、目标地址和客户端地址，适合在浏览器中直接展示。

3.4 HTTPS CONNECT 隧道与可见性边界

HTTPS 客户端首先向代理发送 CONNECT 请求，例如：

```
CONNECT baike.baidu.com:443 HTTP/1.1
```

代理完成域名访问控制后连接目标服务器，向客户端返回 200 Connection Established。随后 `forward_connect()` 使用非阻塞 Socket 和 `select.select()` 在客户端与目标服务器之间双向转发加密字节流。

当前实现不解密 TLS，因此能够看到 CONNECT 中的目标域名与端口，无法看到 HTTPS 加密后的路径、查询参数和网页正文。HTTPS 域名拦截可以正常执行，HTTPS URL 路径和正文关键词过滤不在当前实现范围内。该边界在验收文档和自动测试中有明确验证。

3.5 客户端 IP 与 CIDR 访问控制

客户端访问控制用于决定哪些主机能够使用代理。系统从已建立的 TCP 连接中获取客户端源地址，并在认证、限流和目标规则之前执行检查。

`client_ip_matches()` 使用 `ipaddress.ip_address()` 与 `ipaddress.ip_network()` 判断地址是否命中单个 IP 或 CIDR 网段。规则输入时会进行合法性检查和标准化，例如 `127.0.0.99/24` 会保存为 `127.0.0.0/24`。

`check_client_access_policy()` 先检查 `blocked_client_ips`。命中黑名单时返回 `client_ip_blacklist`。随后检查 `allowed_client_ips`；白名单非空且客户端未命中时返回 `client_ip_not_allowed`。黑名单优先级高于白名单。客户端规则仅作用于代理端口，管理端口仍可用于删除错误规则并恢复访问。

3.6 代理认证与访问频率限制

代理认证使用 HTTP Basic 代理认证机制。启用认证后，系统读取 `Proxy-Authorization` 请求头，完成 Base64 解码并与运行配置中的用户表比较。认证失败时返回 `407 Proxy Authentication Required`，响应头包含 `Proxy-Authenticate`。代理认证头在转发上游请求时会被删除，目标网站不会收到代理凭据。

访问频率限制按客户端 IP 建立固定时间窗口计数桶。每个桶保存当前计数和重置时间。窗口内请求数达到上限后，代理返回 `429 Too Many Requests` 并记录建议等待时间。认证挑战不会消耗正常访问额度；运行时修改限流参数后，旧计数桶会被清空。

3.7 HTTP GET 缓存

缓存键由目标主机、端口和请求路径组成。启用缓存后，GET 请求在连接上游服务器前查询内存缓存。有效缓存命中时直接返回响应，并增加缓存命中统计；未命中时记录缓存未命中事件。上游响应状态为 200 时写入缓存。

每个缓存条目保存响应字节、状态码、创建时间和过期时间。读取时自动删除过期条目，写入超过最大容量时删除最早创建的条目。管理 API 提供缓存清空操作，便于重复展示第一次访问未命中、第二次访问命中的行为。

3.8 动态规则管理与持久化

规则管理提供新增、删除、修改、查询和整组替换操作。管理前端与命令行工具最终调用相同的 JSON API。后端首先检查规则组名称，再根据规则类型对值进行标准化。例如 HTTP 方法统一转换为大写，IP/CIDR 规则使用 `ipaddress` 标准化，域名

规则去除无意义空白。

`RuntimeState` 在锁内更新当前配置，并将完整配置写回启动参数指定的 JSON 文件。规则修改结果包含 `changed`、`saved`、规则组和最新值列表，便于命令行和前端回显。每次变更同时追加到 JSONL 变更日志，代理重启后仍可查询历史操作。

命令行工具 `tools/rule_cli.py` 对管理 API 进行了封装。例如：

```
python tools\rule_cli.py add blocked_domains *.example.com
python tools\rule_cli.py update blocked_url_keywords game private
python tools\rule_cli.py delete blocked_content_keywords forbidden
python tools\rule_cli.py set rate_limit_enabled true
```

3.9 日志审计与运行统计

系统将事件分为访问、拦截和错误三类日志。放行、缓存命中、CONNECT 隧道等事件写入访问日志；域名拒绝、客户端拒绝、正文过滤、认证挑战和限流拒绝写入拦截日志；请求解析错误和上游连接错误写入错误日志。

每条日志包含时间、事件类型和键值字段，例如客户端地址、请求方法、目标主机、路径、状态码、字节数和拦截原因。日志模块能够读取尾部记录，将键值字段解析为结构化数据，并按事件类型和全文关键词筛选。查询结果可以通过管理前端查看，也可以导出为 CSV。

`RuntimeState.snapshot()` 生成实时统计快照，包括总请求、放行、各类拦截、正文过滤、HTTPS 隧道、缓存命中与未命中、认证挑战、限流、错误和传输字节数。管理首页中的统计项可以点击，并跳转到对应日志详情。

3.10 管理前端

管理前端由原生 HTML、CSS 和 JavaScript 实现，通过 Fetch API 调用管理后端。首页显示当前模式、运行设置、规则组、统计卡片、域名排行、关键词命中排行、最近访问日志和规则变更回显。规则支持新增、修改和删除，运行设置支持直接保存。

日志详情页支持选择日志类型、事件和搜索关键词，能够查看当前运行日志或自动验收证据。规则变更详情页支持按操作类型和规则组查询。自动测试生成的证据保存在独立目录，管理首页可以读取最近一次验收结果，便于在测试结束后统一展示。

4 测试方案与验收结果

4.1 测试方法

项目采用单模块测试、批量回归测试和手工验收三种方式。单模块测试针对一个功能创建独立端口、配置和测试目标，能够准确检查返回状态、响应正文、配置保存、统计和日志。批量测试将功能分为 Web/代理功能与规则/运行模式两个批次，两个批次使用不同证据目录，重复运行时不会相互覆盖。手工验收使用本地测试 Web 服务器、命令行请求和独立浏览器配置，便于现场逐项展示。

4.2 自动测试结果

2026 年 6 月 11 日执行完整回归测试，Web/代理功能批次共 15 项，规则与运行模式批次共 7 项，全部通过。

测试批次	覆盖内容	结果
Web/代理功能	HTTP 转发、域名拦截、URL 与方法过滤、正文过滤、HTTPS CONNECT、缓存、前端显示、认证、限流、浏览器代理、公开 HTTP 示例与验收证据	15/15
规则与运行模式	管理 API、白名单模式、规则增删改查、CLI、运行设置热更新、客户端 IP/CIDR 与规则证据	7/7

完整测试命令为：

```
python tests\run_all_smoke.py
```

测试通过时输出 all test batches passed。Web 验收证据保存在 tests/evidence/web，规则验收证据保存在 tests/evidence/rules，管理前端能够读取这些证据并重建统计。

4.3 客户端 IP 黑白名单实测

客户端 IP 模块使用 tests/stage18_client_ip_policy_smoke.py 单独验证。测试先通过代理访问本地上游服务器并获得 200 OK；随后将 127.0.0.99/24 加入黑名单，确认规则被标准化为 127.0.0.0/24，请求被拒绝并记录 client_ip_blacklist；修改黑名单后，访问立即恢复。

白名单测试加入不匹配本机的 10.0.0.0/8, 确认请求被拒绝并记录 `client_ip_not_allowed`; 随后将规则修改为 127.0.0.1, 请求立即恢复。测试还检查非法 IP 规则会返回 400, 并确认客户端拦截统计值与结构化日志记录一致。

4.4 核心功能手工验收

启动本地测试目标和代理:

```
python tests\manual_demo_server.py
python src\proxy.py --config config.example.json
```

普通代理转发:

```
curl.exe -i --noproxy no-host-bypass.invalid ^
-x http://127.0.0.1:8080 http://127.0.0.1:9000/
```

正文过滤和 URL 过滤:

```
python tools\rule_cli.py add blocked_content_keywords forbidden
python tools\rule_cli.py add blocked_url_keywords game

curl.exe -i --noproxy no-host-bypass.invalid ^
-x http://127.0.0.1:8080 http://127.0.0.1:9000/content-test.html

curl.exe -i --noproxy no-host-bypass.invalid ^
-x http://127.0.0.1:8080 http://127.0.0.1:9000/game/index.html
```

客户端黑名单:

```
python tools\rule_cli.py add blocked_client_ips 127.0.0.99/24
curl.exe -i --noproxy no-host-bypass.invalid ^
-x http://127.0.0.1:8080 http://127.0.0.1:9000/
python tools\rule_cli.py delete blocked_client_ips 127.0.0.0/24
```

详细命令参数、预期响应、日志查询和恢复步骤记录在 `docs/08_backend_cli_rule_management`.

5 安全性分析与项目边界

5.1 安全控制效果

代理通过多层策略链对请求进行检查。客户端 IP 规则在认证与限流之前执行, 能够提前拒绝无权使用代理的主机。代理认证限制未授权用户使用代理, 认证头在转发

前被删除。域名、URL 和方法规则在建立上游连接前执行，能够减少被禁止目标的网络访问。正文过滤在收到明文响应后执行，命中后使用本地生成的提示页替代原网页。

日志模块为访问和规则修改提供审计依据。拦截原因采用稳定字段表示，能够区分域名黑名单、客户端黑名单、白名单拒绝、URL 关键词、正文关键词、认证和限流等场景。规则变更持久化后，可以确认操作时间、操作类型、规则组和修改结果。

5.2 现有边界

项目没有实现 HTTPS 中间人解密，因此 HTTPS 路径和正文不可见。该设计避免了自签证书安装、证书信任和私钥保护问题，适合作为课程实验中的普通正向代理。若继续扩展 HTTPS 正文过滤，需要生成并安全保存 CA 证书，为每个目标动态签发证书，并处理客户端信任和合规风险。

正文过滤当前针对 UTF-8 文本和未压缩响应。不同字符编码、分块传输、压缩正文和大文件流式处理仍有改进空间。缓存使用进程内内存，代理重启后缓存会清空。日志使用文件保存，数据量较大时可以迁移到 SQLite，以支持索引、分页和聚合查询。

管理端口默认监听本机地址，当前未增加独立的管理端登录。实际部署时应限制管理端监听范围，并增加身份认证、权限分级和安全传输。

6 总结与展望

本实验完成了 Web 代理服务器的需求分析、总体设计、核心实现和完整验收。系统能够接收并转发 HTTP 代理请求，能够拦截指定域名，能够过滤包含指定关键词的明文网页正文，满足题目提出的基础要求。

项目在基础功能之上增加了 HTTPS CONNECT、URL 与方法过滤、客户端 IP/CIDR 黑白名单、代理认证、访问频率限制、HTTP GET 缓存、动态规则管理、日志审计、统计展示和管理前端。规则可以通过前端、API 和命令行修改，修改结果能够持久化并在前端回显。自动测试和手工验收覆盖了功能结果、配置保存、日志记录和统计展示，使系统具备可重复验证能力。

后续可继续实现管理端认证、SQLite 审计存储、规则快照与回滚、定时策略和更完善的流式响应处理。HTTPS 内容检查需要结合证书管理和安全边界进行专门设计。

A 常用运行与验收命令

命令	用途
python src\proxy .py --config config.example.json	启动代理服务和管理前端。
python tests\manual_demo_server.py	启动本地确定性验收目标。
python tests\run_all_smoke.py	运行完整自动回归测试。
python tools\rule_cli.py list	查询所有规则组。
python tools\rule_cli.py config	查询当前运行配置。
python tools\rule_cli.py changes --limit 20	查询最近规则变更。
python tools\rule_cli.py log-query --kind blocked --limit 20	查询最近拦截日志。

B 主要管理 API

路径	方法	功能
/api/rules	GET	查询可编辑规则组。
/api/rules/add	POST	新增规则。
/api/rules/delete	POST	删除规则。
/api/rules/update	POST	修改单条规则。
/api/rules/replace	POST	替换整个规则组。
/api/settings/update	POST	修改运行设置。
/api/stats	GET	查询实时统计。
/api/logs/query	GET	按条件查询日志。
/api/logs/export.csv	GET	导出日志查询结果。
/api/evidence/dashboard	GET	查询最近自动验收统计。